

Milvus Vector Database

Complete Guide to Indexes

A Student- Friendly Reference

Table of Contents

Part 1 - Basic Concepts of Indexing	4
1.1 Inverted File Index (IVF)	4
1.2 Quantization	5
Scalar Quantization (SQ)	5
Product Quantization (PQ)	5
Binary / RaBitQ Quantization	6
Part 2 - Data Structures	7
2.1 Cluster- Based Structures (IVF Family)	7
What is a cluster?	7
Inverted Lists	7
2.2 Graph- Based Structures (HNSW, DiskANN)	8
What is a navigable graph?	8
HNSW - Hierarchical Navigable Small World	8
Vamana Graph (DiskANN)	8
2.3 LSH - Locality- Sensitive Hashing	8
Part 3 - Float Vector Indexes	9
3.1 FLAT - Brute Force / Exact Search	9
How it works	9
Parameters to tune	9
Pros	9
Cons	9
3.2 IVF_FLAT - Clustered Exact Search	10
How it works	10
Parameters to tune	10
Pros	10
Cons	10

3.3	IVF_SQ8 - IVF with Scalar Quantization.....	10
	How it works	10
	Parameters to tune	11
	Pros.....	11
	Cons.....	11
3.4	IVF_PQ - IVF with Product Quantization	11
	How it works	11
	Parameters to tune	11
	Pros.....	12
	Cons.....	12
3.5	IVF_RABITQ - IVF with Binary Quantization + Refinement.....	12
	How it works	12
	Parameters to tune	12
	Pros.....	13
	Cons.....	13
3.6	HNSW - Hierarchical Navigable Small World.....	13
	How it works	13
	Parameters to tune	14
	Pros.....	14
	Cons.....	14
3.7	HNSW_SQ - HNSW + Scalar Quantization	14
	How it works	14
	Parameters to tune	14
	Pros.....	15
	Cons.....	15
3.8	HNSW_PQ - HNSW + Product Quantization	15
	How it works	15
	Parameters to tune	15
	Pros.....	15
	Cons.....	15
3.9	HNSW_PRQ - HNSW + Product Residual Quantization	16
	How it works	16
	Parameters to tune	16
	Pros.....	16
	Cons.....	16
3.10	SCANN - Google's Scalable Nearest Neighbours.....	16
	How it works	16
	Parameters to tune	17
	Pros.....	17

Cons	17
3.11 DiskANN - Disk- Based Billion- Scale Search	17
How it works	17
Parameters to tune	17
Pros	18
Cons	18
3.12 AISAQ - All- In- Storage ANNS (Milvus 2.6.4+)	18
How it works	18
Parameters to tune	18
Pros	18
Cons	19
3.13 GPU Indexes	19
GPU_CAGRA	19
GPU_IVF_FLAT	19
GPU_IVF_PQ	19
GPU_BRUTE_FORCE	19
3.14 Float Vector Index Comparison Table	20
Part 4 - Binary Vector Indexes	21
Distance Metrics for Binary Vectors	21
4.1 BIN_FLAT - Exact Binary Search	21
How it works	21
Parameters	21
Pros	21
Cons	21
4.2 BIN_IVF_FLAT - Binary IVF (Approximate Search)	22
How it works	22
Parameters to tune	22
Pros	22
Cons	22
4.3 MINHASH_LSH - MinHash Locality- Sensitive Hashing	22
How it works	22
Parameters to tune	23
Pros	23
Cons	23
4.4 Binary Vector Index Comparison Table	23
Part 5 - Scalar Field Indexes	24
5.1 VARCHAR (String) Indexes	24
INVERTED - General Purpose (Recommended default)	24
BITMAP - Low Cardinality Equality	24

Trie (Prefix Tree) - Prefix/Autocomplete Search.....	24
5.2 VARCHAR Index Comparison	24
5.3 Boolean (BOOL) Indexes	25
BITMAP - Strongly Recommended	25
INVERTED - Rarely Needed.....	25
5.4 Integer (INT8, INT16, INT32, INT64) Indexes	25
STL_SORT - Recommended for Range Queries	25
INVERTED - For Exact Match / Categorical Integers	25
5.5 ARRAY Field Indexes.....	26
BITMAP on Arrays	26
INVERTED on Arrays	26
Part 6 - Quick Decision Guide.....	26
6.1 Float Vector Index Decision Guide	26
6.2 Binary Vector Index Decision Guide	27
6.3 Scalar Field Index Quick Reference	27
Glossary - Key Terms at a Glance	28

Part 1 - Basic Concepts of Indexing

Before diving into specific indexes, let's build a clear mental model of the core ideas. These concepts are the building blocks used by nearly every index in Milvus.

1.1 Inverted File Index (IVF)

Imagine you have 10 million vectors (like word embeddings). Searching all of them for every query is very slow. IVF is a clever trick to search only a small fraction of them.

The Core Idea - Cluster, then Search

- Group all vectors into buckets (clusters) using k- means. Each bucket is centered on a representative point called a centroid.
- At query time, find the closest centroids first (fast - only a few thousand centroids exist), then search only the vectors inside those top buckets.
- Result: instead of comparing 10 million vectors, you compare maybe 50,000 - a 200× speedup!

Key IVF Parameters

Parameter	Phase	What it does	Practical tip
nlist	Build	Number of clusters to create (e.g. 1024, 4096, 65536)	More clusters → better recall at query time, but slower build and more RAM
nprobe	Search	How many clusters to look inside when answering a query	Higher nprobe = higher recall but slower query; tune until recall is acceptable

Step-by-Step: How IVF Works

- Step 1 - Build: Run k-means on all your vectors to create nlist centroids. Assign every vector to its nearest centroid and store lists ("inverted lists") of vectors per centroid.
This creates the "inverted" structure: instead of "vector → its position", it's "centroid → list of vectors close to it".
- Step 2 - Query: Compute distance from your query to all centroids (fast). Pick the top nprobe centroids. Scan only the vectors in those clusters. Return the best k results.

 **Real-world analogy:** IVF is like a library with sections (clusters). You check the index card (centroid) to find the right section, then only search books in that section - not the entire library.

1.2 Quantization

Quantization is about storing vectors in a compressed, lossy format to save memory and speed up distance calculations. It trades a small amount of accuracy for a large memory saving.

Why Do We Need It?

- A 768-dimensional float32 vector uses $768 \times 4 = 3,072$ bytes $\approx$ 3 KB per vector.
- One million vectors = ~3 GB. One billion vectors = ~3 TB - too much RAM!
- Quantization shrinks each vector so you can fit billions into modest hardware.

Scalar Quantization (SQ)

What it does: Converts each dimension from a 32-bit float to an 8-bit integer (SQ8) independently.

How: Find the min and max value across a dimension, then map the full float range to 0–255. A float value of 3.25 might become the integer 200.

Compression: 4× smaller (4 bytes → 1 byte per dimension).

Recall impact: Small drop - recall typically stays >90% in IVF_SQ8.

 **When to use SQ:** Your default memory-saving choice when you want simplicity and still need high recall.

Product Quantization (PQ)

What it does: Instead of quantizing each dimension independently, PQ splits the vector into M equal sub-vectors and quantizes each sub-vector using its own small codebook.

- Step 1: Chop a 768-dim vector into M sub-vectors (e.g. 32 sub-vectors of 24 dims each).

- Step 2: For each sub- vector group, run k- means on the training set to find 256 centroids (a "codebook" with 8- bit codes = 256 entries).
- Step 3: Replace each sub- vector with an 8- bit code (which of the 256 centroids it is closest to).
- Result: Store M bytes instead of 4×768 bytes - e.g., 32 bytes instead of 3,072 bytes = 96× compression!

Note: codebook looks like:

Subvector 1 → code 73

Subvector 2 → code 14

Subvector 3 → code 201 ...

Subvector 1 → 24 dimensions array.

[0.15,

0.87,

1.02,

...

0.44]

One of PQ's biggest advantages is that it can compute approximate distances **without reconstructing the full vector**.

Suppose the query is divided into the same 32 subvectors.

For each query subvector, PQ computes its distance to **all 256 centroids** in that subspace

So instead of computing distances between floating-point vectors, the search performs a few **table lookups and additions**, making it much faster.

Why only 8 bits? – 256 clusters, $\log_2(256)=8$ bits. If there are 1024 clusters, 10 bits will be needed to cover all the cluster numbers.

Parameter	Meaning	Effect of increasing
m (subspaces)	How many equal parts to split the vector into	More parts = better accuracy but slower and bigger codebook
nbits	Bits per codebook entry (usually 8 → 256 centroids)	More bits = better accuracy but much more memory for codebook

 **When to use PQ:** Very tight memory budgets (10–64× savings). Accept a bigger recall drop (75–90% recall) vs SQ's ~90–95%.

Binary / RaBitQ Quantization

What it does: Compresses each dimension to a single bit (0 or 1) - the most extreme compression possible (~32× vs float32).

RaBitQ's twist: Instead of a naive sign- bit, RaBitQ uses randomised projections and error bounds from information theory to minimise accuracy loss at 1- bit precision.

- Enables super- fast distance computation using bitwise XOR operations (CPU- friendly).
- Optional refinement step stores extra SQ8/FP16 data to re- rank top candidates and recover near- full recall.

 **When to use Binary/RaBitQ:** Extreme memory savings needed + modern hardware + willing to enable refinement for recall. Available in Milvus 2.6+ as IVF_RABITQ.

Quantization Comparison Summary

Type	Bits/dim	Compression	Recall	Complexity	Milvus Index
SQ (Scalar)	8 bits	~4×	High (90–95%)	Simple	IVF_SQ8, HNSW_SQ
PQ (Product)	0.5–1 bit effective	8–64×	Medium (75–90%)	Moderate	IVF_PQ, HNSW_PQ
RaBitQ (Binary)	1 bit	~32×	Good with refinement (90–96%)	Advanced	IVF_RABITQ

Part 2 - Data Structures

Vector indexes are built on top of two main families of data structures: clusters (partition- based) and graphs (navigation- based). Understanding these will make all the index types intuitive.

2.1 Cluster- Based Structures (IVF Family)

What is a cluster?

A cluster is a group of vectors that are "close" to each other in the embedding space. Think of it as a neighbourhood.

- Each cluster has a centroid - its representative centre point.
- Vectors are assigned to the nearest centroid at build time.
- At search time, you only need to check the clusters whose centroids are closest to your query - ignoring the rest.

 **Mental model:** Imagine a map of a city with neighbourhood centres (centroids). To find a restaurant near you, you check your neighbourhood and maybe 1–2 adjacent ones - not every restaurant in the city.

Inverted Lists

The actual storage structure behind IVF is called an inverted list (or posting list). For each centroid, there is a list of vector IDs (and the vectors themselves) assigned to it.

- Centroid A → [vector_id_5, vector_id_23, vector_id_891, ...]
- Centroid B → [vector_id_2, vector_id_67, ...]

During search, Milvus only reads the inverted lists of the top nprobe centroids - skipping all others.

2.2 Graph- Based Structures (HNSW, DiskANN)

What is a navigable graph?

A graph where every vector is a node, and edges connect nearby (similar) vectors. To search, you start from an entry node and greedily hop to closer and closer nodes until you find the best match.

HNSW - Hierarchical Navigable Small World

HNSW is the most popular graph index for in- memory ANN search. It builds a multi- layer graph:

- Top layers: sparse connections → fast long- range jumps (like highways).
- Bottom layers: dense connections → precise local search (like neighbourhood streets).
- Search: enter at top, jump quickly to the right area, then refine locally at the bottom.

HNSW Parameter	What it controls	Typical value	Trade- off
M	Number of connections per node per layer (graph density) – Index construction	8–64	Higher M = better recall and quality, but more RAM and slower build
efConstruction	Candidates explored when adding a node during build - Index construction	40–512	Higher = better graph quality, much slower build
ef (exploration factor)	Breadth of search while querying - Querying	16–512	Higher = better recall, slower query

 **Analogy:** HNSW is like a navigation app with zoom levels. At the country level (top layer) you find the right city fast. At the street level (bottom layer) you find the exact address.

Vamana Graph (DiskANN)

Used by DiskANN (and AISAQ). Similar to HNSW but designed for disk storage:

- Fewer outgoing edges per node → fewer disk seeks during traversal.
- Beam search instead of greedy best- first → explores multiple paths at once.
- Most of the graph lives on SSD; only PQ- compressed representations stay in RAM.

2.3 LSH - Locality- Sensitive Hashing

Used by MINHASH_LSH for binary/set data. The idea: hash similar items into the same bucket with high probability.

- Apply multiple hash functions to each vector/set.
- Items with similar hashes go into the same bucket.
- At search time: only compare the query against items in the same buckets (huge speedup).

 **Best for:** Set similarity (Jaccard), document deduplication, near- duplicate detection using binary representations.

Part 3 - Float Vector Indexes

Float vectors are the most common type in Milvus - produced by ML embedding models (BERT, CLIP, etc.). They come in three precision flavours:

Type	Bits	Memory/dim	Precision	Typical use
FLOAT32	32	4 bytes	High	Standard training and inference
FLOAT16	16	2 bytes	Medium	Inference with memory savings
BFLOAT16	16	2 bytes	Medium (wide range)	Mixed- precision ML training

All float vector indexes described below work with FLOAT32, FLOAT16, and BFLOAT16 unless noted. BFloat16 is preferred over Float16 in most modern AI contexts because it preserves numerical stability.

3.1 FLAT - Brute Force / Exact Search

How it works

FLAT is the simplest possible approach: store all vectors in their original form and compare every query against every stored vector. No fancy structure - just raw comparison.

- Build: Do nothing (just store vectors). Build time is essentially zero.
- Search: Calculate exact distance from query to every vector. Pick top k.
- Recall: 100% - always finds the true nearest neighbours.

Parameters to tune

FLAT has no index- specific build parameters. At search time, you just specify top k (how many results to return).

Pros

- 100% recall - perfect accuracy, no approximation.
- Zero build time - useful during rapid experimentation.
- No parameter tuning - completely plug- and- play.

Cons

- Slowest query time - scales linearly $O(n)$ with dataset size.
- Completely impractical beyond a few million vectors.
- High memory - stores full- precision vectors for all items.

☑ **Best for:** Small datasets (<1–5 million vectors) where you need exact results and can't tolerate any recall loss. Also excellent as a benchmark baseline.

3.2 IVF_FLAT - Clustered Exact Search

How it works

- Build: Cluster all vectors into nlist groups using k- means. Store full- precision vectors inside each cluster.
- Search: Find the top nprobe closest clusters. Do exact distance comparison only within those clusters. Return top k.
- Key insight: You skip 99–99.9% of the data while still getting near- perfect recall.

Parameters to tune

Parameter	Phase	Recommended range	Tip
nlist	Build	1024–65536	Rule of thumb: $nlist \approx \sqrt{\text{total vectors}}$. E.g., 1M vectors $\rightarrow nlist \approx 1024$
nprobe	Search	8 to nlist	Start with nprobe=16 and increase until recall target is met. Never exceed nlist.

Pros

- Very high recall (95–99%+) with proper nprobe tuning.
- Fast queries on large datasets compared to FLAT.
- No compression - no accuracy penalty from quantization.

Cons

- High memory usage - full- precision vectors stored in clusters.
- Build time is significant (k- means clustering can be slow on huge datasets).
- Slower than quantized variants like IVF_SQ8 or IVF_PQ.

☑ **Best for:** Large datasets where you want the highest possible recall with good query speed. Classic choice when memory is not the primary constraint.

🔗 **Example:** Building a semantic search engine over 50 million product descriptions - IVF_FLAT with nlist=8192 and nprobe=64 gives excellent results.

3.3 IVF_SQ8 - IVF with Scalar Quantization

How it works

Same as IVF_FLAT, but vectors inside each cluster are compressed using 8- bit Scalar Quantization instead of stored in full float32 precision.

- Build: k- means clustering + SQ8 compression of each vector (each dim: 4 bytes → 1 byte).
- Search: Find top nprobe clusters → compare compressed vectors → return top k.
- Memory: ~4× smaller than IVF_FLAT. Queries are faster due to cheaper arithmetic on integers.

Parameters to tune

Parameter	Phase	Recommended range	Tip
nlist	Build	1024–65536	Same as IVF_FLAT - tune based on dataset size
nprobe	Search	8 to nlist	Slightly higher nprobe than IVF_FLAT may be needed to compensate for quantization error

Pros

- ~4× memory savings vs IVF_FLAT.
- Faster query speed (integer math is cheaper than float math).
- Recall stays high (~90–95%) - SQ8 is a gentle compression.

Cons

- Slight recall drop compared to IVF_FLAT (acceptable for most applications).
- More complex to tune than FLAT but still simple.

✓ **Best for:** Memory- constrained servers with large datasets. The classic 'best bang for buck' choice - 4× memory savings with minimal recall loss.

🔗 **Example:** A RAG system with 100M document chunks on a 64GB server - IVF_SQ8 lets you fit the whole index in RAM where IVF_FLAT would not.

3.4 IVF_PQ - IVF with Product Quantization

How it works

Combines IVF clustering with Product Quantization for aggressive compression. Instead of quantizing each dimension individually (SQ), PQ splits the vector into M subspaces and quantizes each subspace with its own codebook.

- Build: Cluster vectors with k- means (nlist). For each cluster, further compress vectors using PQ (m subspaces, nbits per codebook entry).
- Search: Find top nprobe clusters → use precomputed distance lookup tables for PQ codes → return top k.
- Memory: 8–64× smaller than IVF_FLAT.

Parameters to tune

Parameter	Phase	Recommended range	Tip
nlist	Build	1024–65536	Standard IVF clustering parameter
m	Build	dim/4 to dim/2	More sub- vectors = better accuracy but more codebook memory. Must divide dim evenly.
nbits	Build	8 (usually fixed)	8 bits = 256 centroids per subspace. Increasing to 12 improves accuracy but quadruples codebook size.
nprobe	Search	8 to nlist	May need higher values than IVF_SQ8 due to larger compression error.

Pros

- Very strong compression - 8 to 64× smaller than full- precision.
- Extremely fast queries due to precomputed lookup table distances.
- Scalable to billions of vectors on commodity hardware.

Cons

- Larger recall drop (75–90%) compared to SQ8 - needs careful tuning.
- More parameters to tune (m, nbits) - not beginner- friendly.
- Codebook training adds to build time.

✓ **Best for:** Very large collections (hundreds of millions to billions of vectors) with very tight memory budgets where you can accept a moderate recall drop.

🔗 **Example:** A video platform doing image- similarity search over 500M thumbnail embeddings on a server with only 32GB RAM - IVF_PQ is the only in- memory option.

3.5 IVF_RABITQ - IVF with Binary Quantization + Refinement

How it works

The newest and most advanced IVF index (Milvus 2.6+). Uses RaBitQ to compress each vector to approximately 1 bit per dimension - the most extreme compression available.

Two- Stage Search Process:

- Stage 1 (Coarse, ultra- fast): Probe nprobe clusters. Within each cluster, compute approximate distances using binary XOR/bit operations on 1- bit codes. Keep a large candidate list (topK × expansion_factor).
- Stage 2 (Refinement, optional): For the surviving candidates, reload their higher- precision representation (SQ8, FP16, etc.) and recompute accurate distances. Re- sort and return final top k.

Parameters to tune

Parameter	Phase	Recommended	Tip
nlist	Build	1024–65536	Standard IVF parameter
refine	Build	true/false	Always set true in production - huge recall boost
refine_type	Build	SQ8 (default)	SQ8 = best balance. FP32 = max recall. SQ6 = most compact.
nprobe	Search	8 to nlist	Can be lower than IVF_PQ because refinement recovers recall
expansion_rate	Search	10–100	How many extra candidates to collect before refinement

Refinement Type Memory Trade- offs (768- dim vectors):

Config	Total Memory vs IVF_FLAT	Typical Recall	QPS vs IVF_FLAT
IVF_RABITQ (no refine)	~3–5%	75–88%	3–5× faster
IVF_RABITQ + SQ8 refine	~25–30%	93–96%	2.5–4× faster
IVF_RABITQ + FP32 refine	~100%+	~99–100%	1.5–2.5× faster
IVF_FLAT (baseline)	100%	95–99%	1× (baseline)

Pros

- Extreme memory savings (~32× vs float32 for base index).
- Best QPS of any IVF variant - binary operations are very CPU- efficient.
- With SQ8 refinement: achieves near- IVF_FLAT recall at ~25% of the memory.

Cons

- Only available in Milvus 2.6+ - not supported in older versions.
- Refinement adds storage overhead (though still much cheaper than IVF_FLAT).
- More parameters to understand (refine, refine_type, expansion_rate).

✓ **Best for:** High- QPS systems with memory constraints where you want modern state- of- the- art compression. Ideal for billion- scale RAG, recommendation, or real- time search.

🔗 **Example:** A recommendation engine for 1B product embeddings - IVF_RABITQ + SQ8 refinement uses only ~25% of IVF_FLAT's memory while delivering similar recall at 3× the throughput.

3.6 HNSW - Hierarchical Navigable Small World

How it works

HNSW is a graph- based index considered the gold standard for in- memory high- recall ANN search. It builds a multi- layer navigable graph where higher layers have fewer, longer- range connections and lower layers have dense local connections.

- Build: Insert each vector as a node. For each insertion, greedily connect it to M nearest neighbours at each layer.
- Search: Enter graph at top layer → greedily jump to closest nodes → descend layers → local refinement at bottom layer. ef controls how many candidates to explore at each step.
- No clustering needed - the graph itself organises the space.

Parameters to tune

Parameter	Phase	Recommended	Tip
M	Build	8–64	M=16 is a safe default. M=32–64 for max recall. More M = more RAM.
efConstruction	Build	40–512	200 is a good starting point. Higher = better graph = slower build.
ef	Search	16–512	Must be >= topK. Increase until recall target is hit.

Pros

- Best recall among all indexes (95–99%+) without any quantization loss.
- Very fast queries - graph traversal is extremely efficient.
- No nlist/nprobe - simpler conceptually (just tune M and ef).

Cons

- High memory - stores full vectors + graph structure (~1.5–2× raw data size).
- Slow build time for very large datasets.
- Difficult to scale beyond available RAM - no disk- spillover in basic HNSW.

☑ **Best for:** Latency- sensitive applications (RAG, real- time semantic search) where you need the highest recall and have sufficient RAM. The 'classic' production choice.

🔗 **Example:** A customer support chatbot doing real- time semantic search over 10M FAQ embeddings - HNSW with M=32, ef=128 delivers sub- millisecond high- recall search.

3.7 HNSW_SQ - HNSW + Scalar Quantization

How it works

Same graph structure as HNSW, but vectors stored at graph nodes are compressed using 8- bit Scalar Quantization instead of full float32. The graph topology is unchanged - only the stored vectors are compressed.

Parameters to tune

- All HNSW parameters (M, efConstruction, ef) - same meaning.
- SQ bits: typically fixed at 8 (SQ8). Some variants allow 6- bit.

Pros

- ~75% smaller memory than HNSW (SQ8 = 1 byte/dim vs 4 bytes/dim).
- Maintains most of HNSW's recall (90–96%).
- Simple - only one extra concept beyond HNSW (SQ compression).

Cons

- Slight recall drop vs plain HNSW - quantization introduces error.
- Still memory- heavy if dataset is very large (just not as bad as HNSW).

☒ **Best for:** Teams that like HNSW's simplicity and high recall but are running into memory limits. A great middle ground.

3.8 HNSW_PQ - HNSW + Product Quantization

How it works

HNSW graph with PQ- compressed vectors at each node. Provides much stronger compression than HNSW_SQ but with a larger recall drop.

Parameters to tune

- M, efConstruction, ef - HNSW graph parameters.
- m (PQ subspaces) - more subspaces = better quality.
- nbits - usually 8 (fixed).

Pros

- Strong compression (8–32× vs raw data).
- Good query speed from graph traversal.

Cons

- Larger recall drop (80–93%) - PQ introduces more error than SQ.
- More parameters to tune - less beginner- friendly.

☒ **Best for:** Memory- constrained environments where HNSW_SQ is still too large and you need the graph's speed advantage.

3.9 HNSW_PRQ - HNSW + Product Residual Quantization

How it works

An upgraded version of HNSW_PQ that adds residual quantization stages. After each PQ step, it quantizes the remaining 'error' (residual) with another codebook - improving accuracy at the same compression level.

- Stage 1: Standard PQ on the original vector.
- Stage 2: Compute residual = original vector – reconstructed PQ vector.
- Stage 3: Quantize the residual with another small PQ codebook.
- Repeat for nrq stages (usually 1–3). Each stage captures finer detail missed by the previous.

Parameters to tune

Parameter	What it does	Typical value
M	HNSW connections per layer	16–64
efConstruction	Build- time candidates	100–512
m	PQ subspaces	dim/4 to dim/2
nbits	Bits per codebook entry	8
nrq	Number of residual stages	1–3 (1 is most common)

Pros

- Better recall vs HNSW_PQ at same compression level.
- Best size vs accuracy trade- off in the HNSW family.
- Optional refinement (refine: true) can push recall near HNSW levels.

Cons

- More complex than HNSW_PQ - extra nrq parameter.
- Slightly larger index size vs plain HNSW_PQ (extra residual codebooks).
- Slower build time than HNSW_PQ.

☒ **Best for:** Billion- scale RAG or image/text search where you need the best memory- efficiency you can get while keeping recall above 88–94%.

3.10 SCANN - Google's Scalable Nearest Neighbours

How it works

SCANN is Google's open- source ANN library integrated into Milvus. It builds on IVF + PQ but with key innovations for better accuracy at aggressive compression:

- Anisotropic clustering: Unlike standard k- means, SCANN's clustering minimises error in directions that matter most for ranking (parallel to query vectors) rather than simple reconstruction error.

- Score- aware PQ: Product Quantization is trained to preserve dot- product ranking accuracy, not just vector reconstruction.
- FastScan: SIMD- optimized lookup tables for extremely fast distance computation.
- SOAR (newer): 'Spilling' where vectors can belong to multiple clusters with orthogonal residuals - reduces quantization error without extra overhead.

Parameters to tune

- nlist: Number of clusters (IVF partition count).
- reorder: Whether to re- rank top candidates using full- precision distances.
- quantization_bits: Compression level (4–8 bits per sub- dimension).

Pros

- Often outperforms HNSW in throughput (QPS) by 20–30% on CPU.
- Very high recall (92–98%) with smart quantization.
- Memory- efficient (4–16× savings tunable).

Cons

- CPU- only - no GPU acceleration.
- Less tunable/documented than HNSW in Milvus (fewer knobs exposed).
- Build time is moderate to slow.

☒ **Best for:** High- throughput CPU workloads with large datasets. Teams wanting Google- scale efficiency without GPUs.

3.11 DiskANN - Disk- Based Billion- Scale Search

How it works

DiskANN (developed by Microsoft Research) solves the problem of billion- scale search when the full dataset cannot fit in RAM. It stores the graph and full vectors on SSD/NVMe, with only PQ- compressed representations in RAM.

- Build: Create a Vamana graph (disk- friendly navigable graph) and store it on SSD. Compress all vectors with PQ and keep those PQ codes in RAM only.
- Search: Traverse the graph using beam search. Use in- RAM PQ codes for fast approximate distance estimation. For top candidates, load their full- precision vectors from disk. Refine and return top k.

Parameters to tune

Parameter	What it controls	Typical range	Tip
max_degree	Max outgoing edges per node in Vamana graph	32–512 (default ~56)	Higher = better recall, bigger index, slower build

Parameter	What it controls	Typical range	Tip
search_list_size	Candidate list size during search	100–1000+	Higher = better recall, more disk I/O
beam_width	Width of beam search (parallel paths)	4–16	Higher = better recall, more latency
pq_bits / m	PQ compression level for in- RAM codes	dim/4 to dim/2, 8 bits	Tighter PQ = less RAM but lower recall

Pros

- Scales to billions of vectors on a single machine with limited RAM.
- High recall (90–97%) despite mostly disk- resident data.
- Cost- effective - uses cheap SSD storage instead of expensive RAM.

Cons

- Higher latency than in- memory indexes (5–50ms vs <1ms).
- Requires fast SSD/NVMe - performance collapses on spinning disks.
- Complex setup - must configure disk mode in Milvus (queryNode.enableDisk: true).

 **Best for:** Billion- scale datasets where RAM is the bottleneck. Cost- effective RAG, recommendation, or semantic search at massive scale on commodity hardware.

 **Example:** A legal document search engine over 500M paragraph embeddings - the full index would need ~2TB of RAM in HNSW, but DiskANN fits in 64GB RAM + 2TB SSD.

3.12 AISAQ - All- In- Storage ANNS (Milvus 2.6.4+)

How it works

AISAQ extends DiskANN further by moving even the PQ codes from RAM onto disk. In scale mode, DRAM usage drops to near- zero.

- Base: Vamana graph (like DiskANN) stored on disk.
- Key twist: PQ- compressed codes are also stored on SSD (not in RAM).
- During search: PQ codes are loaded on- demand from disk as needed during traversal.
- Two modes: Performance mode (some PQ codes inline in RAM) vs Scale mode (everything on disk, minimal DRAM).

Parameters to tune

- Storage mode: Performance (lower latency, some RAM) vs Scale (minimal RAM, higher latency).
- PQ parameters (m, nbits) - same as other PQ- based indexes.
- Graph parameters similar to DiskANN.

Pros

- Near- zero DRAM usage in scale mode - extremely cost- effective for massive data.
- Good recall (90–96%) with proper tuning.
- Scales to billions on commodity hardware with fast SSDs.

Cons

- Higher latency than DiskANN (more SSD reads).
- Relatively new - less community documentation and benchmarks.
- Requires Milvus 2.6.4+ and fast NVMe storage.

☑ **Best for:** Ultra- large scale scenarios (billions of vectors) where even DiskANN's RAM usage is too high. Cost- sensitive cloud deployments.

3.13 GPU Indexes

Milvus supports GPU- accelerated indexes for dramatically higher throughput. Requires NVIDIA GPU hardware (CUDA).

GPU_CAGRA

NVIDIA's state- of- the- art GPU graph index (CAGRA = Compute- Adaptive Graph- based index for Rapid Approximate- nearest- neighbor search). Fastest available GPU index.

- Stores full vectors + graph in GPU VRAM.
- Key params: `intermediate_graph_degree` (32–128), `graph_degree`.
- Query speed: 5–20× faster than CPU HNSW.

☑ **Best for:** High- throughput, low- latency GPU workloads where VRAM is available.

GPU_IVF_FLAT

GPU- accelerated version of IVF_FLAT. Stores full vectors in GPU VRAM.

- Params: `nlist`, `nprobe` - same as CPU IVF_FLAT.
- Note: top- k is limited to ≤ 2048 in some versions.

☑ **Best for:** GPU acceleration with high recall needs and moderate dataset sizes.

GPU_IVF_PQ

GPU- accelerated IVF + PQ. Compressed storage in GPU VRAM - allows larger datasets to fit on GPU.

- Higher top- k support than GPU_IVF_FLAT (up to ≤ 8192).

☑ **Best for:** GPU acceleration + memory- constrained scenarios needing higher top- k.

GPU_BRUTE_FORCE

Exact GPU- accelerated search - like FLAT but on GPU. Surprisingly fast for small- medium datasets.

- No build required. 100% recall. No parameters.

☒ **Best for:** Small- medium data (<10–50M vectors) where you need 100% recall and have GPU hardware.

3.14 Float Vector Index Comparison Table

Index	Data Structure	Compression	Recall	QPS	Build Time	Memory	Best For
FLAT	None (raw)	None	100% (exact)	Lowest	None	Full	Small data, exact search
IVF_FLAT	Clusters	None	Very high (95–99%)	Fast	Medium	Full	Large data, high recall
IVF_SQ8	Clusters + SQ	~4×	High (90–95%)	Very fast	Medium	~25%	Memory- limited, good recall
IVF_PQ	Clusters + PQ	8–64×	Medium (75–90%)	Extreme	Slow	~2–12%	Huge data, tight memory
IVF_RABITQ	Clusters + 1-bit	~32×	Good+refine (90–96%)	Extreme	Medium	~3–30%	Max QPS + min memory (Milvus 2.6+)
HNSW	Graph	None	Very high (96–99%)	Very high	Medium-Slow	~150–200%	Latency- sensitive, high recall
HNSW_SQ	Graph + SQ	~4×	High (90–96%)	Very high	Medium	~25%	HNSW with memory savings
HNSW_PQ	Graph + PQ	8–32×	Medium (80–93%)	High	Slow	~5–15%	Tight memory, graph speed
HNSW_PRQ	Graph + PQ+Residual	10–40×	Med- High (88–96%)	High	Slow	~3–12%	Best compression+recall in HNSW family
SCANN	Clusters + adv PQ	4–16×	Very high (92–98%)	Very high	Med-Slow	~6–25%	High CPU throughput
DiskANN	Graph on disk + PQ	~10–20% in RAM	High (90–97%)	Medium (5–50ms)	Slow	Low RAM	Billion+ scale, RAM bottleneck
AISAQ	Graph+PQ on disk	Near- zero RAM	Good (90–96%)	Medium	Slow	Near- zero RAM	Ultra- large scale, cost- sensitive
GPU_CAGRA	Graph (GPU)	~1.8× VRAM	Very high	Extreme (5–20×	Med- Fast	GPU VRAM	GPU, max throughput
GPU_IVF_FLAT	Clusters (GPU)	None	Very high	Extreme	Fast	GPU VRAM	GPU, high recall
GPU_IVF_PQ	Clusters+PQ (GPU)	Tunable	Medium- High	Extreme	Fast	GPU VRAM	GPU + memory limited
GPU_BRUTE_FORCE	None (GPU)	None	100% (exact)	Very high (GPU)	None	GPU VRAM	Small data, GPU, exact

Part 4 - Binary Vector Indexes

Binary vectors store each dimension as a single bit (0 or 1). They are much more compact than float vectors and are ideal for similarity search over hashes, fingerprints, or set- membership representations.

Common use cases: perceptual image hashes (pHash), binary document fingerprints, near- duplicate detection, MinHash- based document deduplication.

Distance Metrics for Binary Vectors

Two standard metrics are used with binary data:

Metric	Measures	Best for	Formula concept
Hamming Distance	Number of bit positions that differ	pHash, binarized embeddings	Count of XOR=1 bits between A and B
Jaccard Similarity	Overlap ratio of 1- bits (set overlap)	Bag- of- words, MinHash, document sets	$(A \text{ AND } B \text{ count}) / (A \text{ OR } B \text{ count})$

4.1 BIN_FLAT - Exact Binary Search

How it works

Exact exhaustive scan over all binary vectors using Hamming or Jaccard distance. Checks every stored vector for every query - no approximation.

Parameters

None (no index- specific build parameters).

Pros

- 100% recall - exact nearest neighbours guaranteed.
- Zero build time.
- Simple - no parameters to tune.

Cons

- Linear scan - slow on large datasets.
- Not practical beyond a few million binary vectors.

☒ **Best for:** Small binary datasets or as a ground- truth benchmark to evaluate approximate binary indexes.

 **Example:** Checking near- duplicate image hashes in a small media library of 100,000 images.

4.2 BIN_IVF_FLAT - Binary IVF (Approximate Search)

How it works

IVF- style clustering applied to binary vectors. Same concept as IVF_FLAT but everything operates in Hamming/Jaccard space instead of Euclidean/cosine.

- Build: Cluster binary vectors into nlist groups (centroids computed in binary space).
- Search: Find top nprobe clusters → exact comparison within those clusters only.

Parameters to tune

Parameter	Phase	Recommended range	Tip
nlist	Build	128–4096	Smaller than float IVF since binary dims are very compact
nprobe	Search	8 to nlist	Higher nprobe = better recall (at cost of speed)

Pros

- Much faster than BIN_FLAT on medium/large datasets.
- High recall achievable with proper nprobe tuning.
- Works with both Hamming and Jaccard metrics.

Cons

- Approximate - may miss some true nearest neighbours.
- Requires nlist/nprobe tuning - more setup than BIN_FLAT.

 **Best for:** Binary datasets up to ~10M vectors needing faster queries. General ANN on binary embeddings: image hashes, binary document signatures.

 **Example:** Finding visually similar product images by comparing perceptual hashes over 5M images.

4.3 MINHASH_LSH - MinHash Locality- Sensitive Hashing

How it works

Designed specifically for set- similarity (Jaccard) over binary data. Combines MinHash with Locality-Sensitive Hashing (LSH):

- MinHash: Apply K independent hash functions to each binary vector (treated as a set of active bit positions). The minimum hash value from each function creates a short signature that preserves Jaccard similarity.
- LSH bucketing: Group similar signatures into the same buckets using band hashing (a technique where signatures are divided into bands and hashed per band).
- Search: Hash the query to buckets → only compare against candidates in the same or nearby buckets → verify true Jaccard on the reduced set.

Parameters to tune

- Number of hash functions K - more = better accuracy, more storage.
- Number of bands (b) and rows per band (r) - control precision/recall tradeoff.
- (Exact parameters depend on Milvus version - check current docs.)

Pros

- Ideal for Jaccard similarity - uses the mathematical structure of sets directly.
- Scales efficiently to very large binary/set collections.
- Great for deduplication - finds near- duplicate documents/images quickly.

Cons

- Approximate by design - probabilistic recall.
- Only suitable for Jaccard- based similarity (not Hamming/Euclidean).
- More complex parameter space than BIN_IVF_FLAT.

☑ **Best for:** Document/text deduplication, near- duplicate detection, set- similarity search at large scale where Jaccard is the natural metric.

🔗 **Example:** Removing duplicate articles from a news aggregator by comparing MinHash fingerprints over 50M documents.

4.4 Binary Vector Index Comparison Table

Index	Search Type	Metric	Recall	Speed	Parameters	Best For
BIN_FLAT	Exact (full scan)	Hamming, Jaccard	100%	Slow (linear)	None	Small data, ground truth
BIN_IVF_FLAT	Approximate (IVF)	Hamming, Jaccard	High (tunable)	Fast (large data)	nlist, nprobe	Medium/large binary ANN
MINHASH_LSH	Approximate (LSH)	Jaccard (sets)	Probabilistic	Very fast (large data)	Hash funcs, bands, rows	Dedup, doc similarity, Jaccard search

Part 5 - Scalar Field Indexes

In Milvus, scalar indexes are used on non- vector fields (strings, numbers, booleans, arrays) to speed up metadata filtering. These are essential in hybrid search: filter on metadata first, then do vector search on the reduced set.

5.1 VARCHAR (String) Indexes

INVERTED - General Purpose (Recommended default)

Builds an inverted index: for each unique term/token in the string values, stores a sorted list of entity IDs containing that term.

- Excellent for: equality filters (field == "AI"), prefix matching (field LIKE "AI%"), multi- term fields ("machine learning" tokenised to [machine, learning]).
- Auto- tokenises strings - no manual preprocessing needed.
- Works for any string cardinality.

☒ **Best for:** General- purpose string filtering - the right choice for 80% of use cases.

BITMAP - Low Cardinality Equality

Stores a compact bitset for each unique string value. Bit i = 1 means entity i has that value.

- Extremely fast for equality checks on fields with few unique values (<1000 unique values).
- Supports bitwise AND/OR across multiple fields for complex filters.
- Inefficient for high- cardinality or prefix searches.

☒ **Best for:** Categorical fields like status ("active"/"inactive"), tier ("free"/"pro"/"enterprise"), country codes.

Trie (Prefix Tree) - Prefix/Autocomplete Search

Builds a character- level tree where shared prefixes share nodes. Paths from root to leaf spell out complete strings.

- Extremely fast prefix queries (LIKE "user- abc%") - just traverse the tree down the prefix path.
- Memory- efficient for fields with many common prefixes.
- Weaker on full equality or non- prefix filters.

☒ **Best for:** Autocomplete, ID prefix lookups, hierarchical identifiers like "prod- v1.2.3" or "user- abc- 123".

5.2 VARCHAR Index Comparison

Index	Best For	Cardinality Fit	Query Types	Memory
INVERTED	General filtering, multi-term	Any	==, LIKE prefix, term list	Moderate
BITMAP	Categorical equality	Low (<1k unique values)	==, AND/OR bitwise	Very low (bitsets)
Trie	Prefix/autocomplete	Any	LIKE prefix%	Low (prefix sharing)

5.3 Boolean (BOOL) Indexes

Boolean fields only have two values: true and false.

BITMAP - Strongly Recommended

One bitset per value (true and false). For 1M entities, the index is only ~125 KB. Equality checks (bool_field == true) become instant bitset scans.

☒ Always use BITMAP for bool fields in production. D6E4F0

INVERTED - Rarely Needed

Stores explicit ID lists for 'true' and 'false'. Much higher memory overhead than BITMAP for only 2 possible values. Only consider if you specifically need term- list compatibility.

5.4 Integer (INT8, INT16, INT32, INT64) Indexes

STL_SORT - Recommended for Range Queries

Builds a sorted array of all integer values with pointers to entity IDs. Range queries (int_field > 100 AND int_field <= 200) use binary search $O(\log n)$ to find the matching range instantly.

- Excellent for: ages, timestamps, scores, ratings, prices, any comparison operators.
- Binary search gives $O(\log N)$ range pruning vs INVERTED's linear term scanning.

INVERTED - For Exact Match / Categorical Integers

Treats each integer as a term. Maps value → list of entity IDs. Fast equality lookups but poor for ranges.

- Use only when integers are categorical enums (e.g., user_type: 1=admin, 2=user, 3=guest).

Index	Best For	Range Query	Equality	Memory
STL_SORT	Range filters, comparisons (>, <, BETWEEN)	Excellent (binary search)	Good	Sorted array
INVERTED	Categorical integers (exact match only)	Poor	Excellent	ID lists

 **Pro tip:** Default to STL_SORT for any numerical field. Use INVERTED only for integer fields that behave like enums.

5.5 ARRAY Field Indexes

Array fields store multiple values per entity. Indexes are built across all array elements in the collection.

BITMAP on Arrays

For each unique element value, creates a bitmap where bit $i = 1$ if entity i 's array contains that value. Query '42 in int_array' = scan bitmap for value 42 - instant!

Ideal for low- cardinality elements: tags, categories, status flags.

INVERTED on Arrays

Same as INVERTED on scalar fields but applied to array elements. Supports more query types (prefix, range on array elements) and handles high- cardinality elements better.

Useful for VARCHAR arrays where prefix filtering on elements is needed.

 **Default choice:** Use BITMAP for arrays of BOOLS, INTs, and low- cardinality VARCHARs. Use INVERTED for high- cardinality elements or when prefix/range filtering on elements is needed.

Part 6 - Quick Decision Guide

Use this section to quickly pick the right index for your situation. Don't overthink it - start with these recommendations and tune from there.

6.1 Float Vector Index Decision Guide

Your Situation	Recommended Index	Why
Small data (<5M vectors), need 100% recall	FLAT	Simple, exact, no surprises
Large data, want highest possible recall	IVF_FLAT	Fast with near- perfect accuracy
Large data + memory is somewhat limited	IVF_SQ8	4× memory savings, still very high recall
Very large data + very tight memory budget	IVF_PQ	Strong compression - tune m and nlist carefully

Your Situation	Recommended Index	Why
Max QPS + min memory, Milvus 2.6+	IVF_RABITQ + SQ8 refine	Best modern binary quantization + fast recall
Need low latency, have RAM, high recall required	HNSW	Gold standard for in- memory latency-sensitive search
Like HNSW but need to save memory	HNSW_SQ	~4× less RAM, minimal recall drop
Tight memory, still want graph speed	HNSW_PQ or HNSW_PRQ	PRQ gives better accuracy vs PQ at same size
High CPU throughput (not GPU)	SCANN	Often beats HNSW on QPS by 20–30%
Billions of vectors, RAM is the bottleneck	DiskANN	Stores graph on SSD, only PQ codes in RAM
Billions of vectors, almost no RAM available	AISAQ	Near- zero DRAM - everything on fast SSD
GPU available, max throughput needed	GPU_CAGRA	NVIDIA's fastest GPU graph index
GPU + need exact results on small-medium data	GPU_BRUTE_FORCE	Exact GPU search, surprisingly fast <50M vecs

6.2 Binary Vector Index Decision Guide

Your Situation	Recommended Index
Small binary dataset, need exact results	BIN_FLAT
Medium/large binary dataset, general ANN	BIN_IVF_FLAT (tune nlist, nprobe)
Large scale, Jaccard similarity, dedup/set similarity	MINHASH_LSH

6.3 Scalar Field Index Quick Reference

Field Type	Use Case	Recommended Index
VARCHAR	General filtering, multi- token strings	INVERTED
VARCHAR	Categorical values (<1000 unique)	BITMAP
VARCHAR	Prefix / autocomplete search	Trie
BOOL	Any boolean filtering	BITMAP
INT (any)	Range queries, comparisons	STL_SORT
INT (any)	Categorical / enum integers, exact match	INVERTED
ARRAY (bool/int/low-cardinality varchar)	Membership queries (42 in array)	BITMAP
ARRAY (high-cardinality / varchar with prefix)	Element prefix or range filters	INVERTED

Glossary - Key Terms at a Glance

Term	Simple Explanation
ANN (Approximate Nearest Neighbour)	Finding vectors that are very close to a query, but not guaranteed to be the absolute closest. Trades a tiny bit of accuracy for huge speed gains.
Centroid	The centre point of a cluster - the 'representative' vector for a group of similar vectors.
Codebook (PQ)	A small set of representative vectors (usually 256) used to approximate sub- vectors in Product Quantization.
Embedding	A vector representation of an item (text, image, etc.) produced by an ML model.
IVF	Inverted File index - partitions vectors into clusters and searches only relevant clusters.
nlist	Number of clusters created during IVF index building.
nprobe	Number of clusters searched during an IVF query. Higher = better recall but slower.
Quantization	Compressing vectors to use fewer bytes, at the cost of a small accuracy drop.
Recall@k	Percentage of true top- k nearest neighbours that appear in your returned results. Higher is better.
QPS	Queries Per Second - how many searches the system can handle per second.
HNSW	Hierarchical Navigable Small World - a multi- layer graph index for fast high- recall search.
Refinement (RABITQ)	An optional second pass that re- ranks candidates using higher- precision data to boost recall.
Vamana graph	A disk- friendly navigable graph used by DiskANN - similar to HNSW but designed for SSD storage.
Hamming distance	Count of bits that differ between two binary vectors. Lower = more similar.
Jaccard similarity	Ratio of shared elements to total elements between two sets. Higher = more similar.
MinHash	A technique that creates a short 'signature' for a set that preserves Jaccard similarity.
LSH	Locality- Sensitive Hashing - groups similar items into the same hash buckets with high probability.